

ICLR 2024 | Denoising Table QA: Weight the Cells, Don't Delete Them

Adobe's MDSR Lab and IIT introduce CABINET, a differentiable relevance-scoring framework that sets new state of the art on three table-QA benchmarks with a 560M-parameter model

Answering a question over a table looks like it should play to a language model's strengths: the structure is explicit, and the answer usually sits in a handful of cells. The hard part is finding those cells. For any given question, most of a table is irrelevant, and that irrelevant content does not just sit there quietly, it behaves like noise. Large language models are known to be susceptible to exactly this kind of distraction, so their table reasoning degrades, and it degrades further as tables grow and carry even more irrelevant data.

One intuitive fix is to shrink the table before answering. Methods like DATER do this by decomposing a large table into a smaller sub-table and reasoning over that. The trouble is that hard decomposition is unforgiving: if the wrong sub-table is extracted, useful information is permanently gone, and no amount of downstream reasoning can recover it.

CABINET (Content Relevance-Based Noise Reduction), from researchers at Adobe's MDSR Lab, IIT Kharagpur, and IIT Roorkee and published at ICLR 2024, takes a gentler route. Instead of removing anything, it learns a relevance weight for every part of the table, amplifying what matters and damping what does not. The answering model always sees the full table; it is simply steered toward the cells that count.

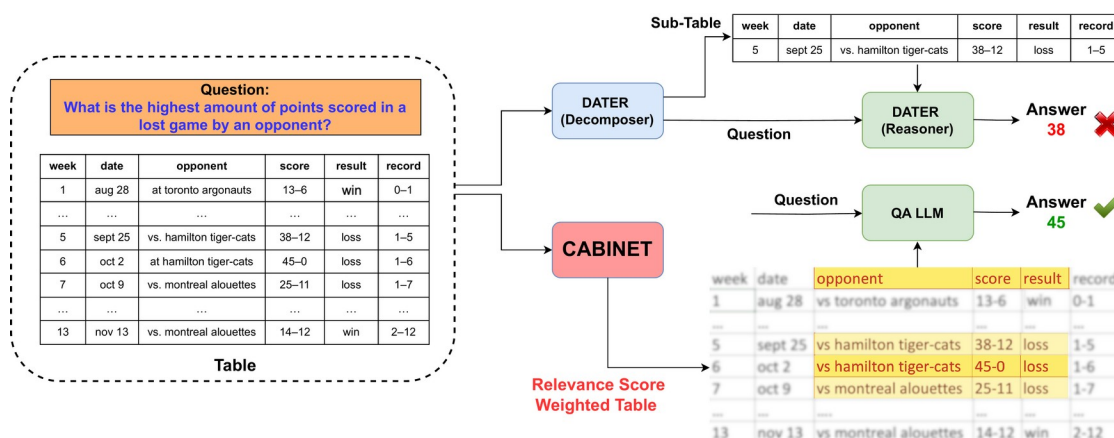


Figure 1. CABINET versus DATER on the same question. DATER hard-decomposes the table into the wrong sub-table (losing useful information) and its reasoner answers incorrectly (38). CABINET removes nothing, weights the relevant table parts higher, and the QA LLM answers correctly (45).

Paper: <https://arxiv.org/abs/2402.01155>

Code: https://github.com/Sohanpatnaik106/CABINET_QA

The problem: irrelevant cells are noise

A table organizes information across rows and columns, but for any single question only a few cells actually contain the answer. Everything else is irrelevant to that question and acts as noise. Prior work has shown that LLMs are easily thrown off by such distracting content, and the effect worsens on large tables because they carry more of it. This vulnerability is precisely what CABINET is built to address.

The case against hard decomposition is a case for keeping information available. If a cell is merely under-weighted rather than deleted, the model can still recover it when needed. That is the bet CABINET makes: soft relevance weighting over irreversible removal.

The method: a relevance score for every token

The pipeline reads left to right in the figure below. The table is linearized into a string with header ([HEAD]) and row ([ROW]) markers, concatenated with the question, and passed through the QA LLM's embedding layer (steps 1 and 2). An Unsupervised Relevance Scorer (URS) then reads the embedded sequence and predicts a relevance score for each table token (step 3). In parallel, a Parsing Statement Generator describes, in natural language, which rows and columns are relevant to the answer, and a cell highlighter turns that description into a cell-based relevance score (steps 4 and 5). The two scores are linearly fused (step 6), used to weight the table content (step 7), and the QA LLM decodes the answer (step 8).

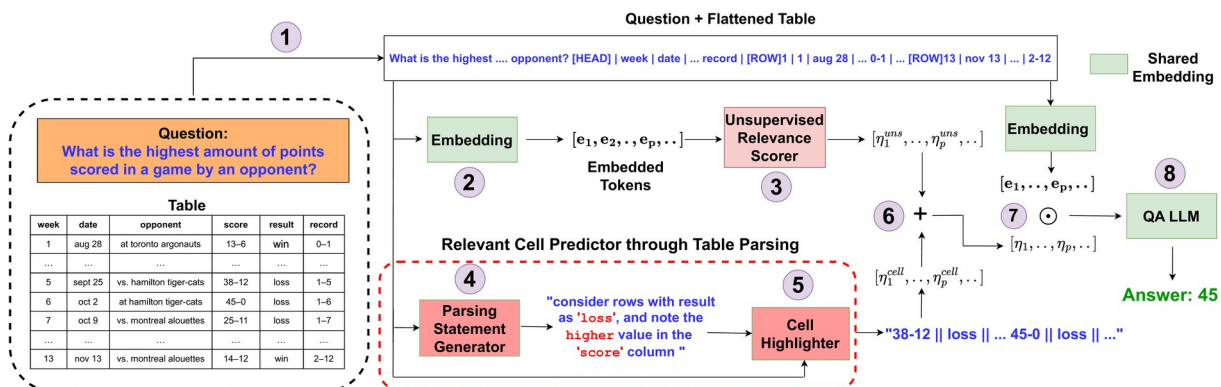


Figure 2. CABINET's architecture. The top path is the Unsupervised Relevance Scorer (URS), which scores every table token; the bottom path is a weakly-supervised Parsing Statement Generator plus Cell Highlighter that produces a cell-based score. The two are fused, applied to weight the table, and passed to the QA LLM (OmniTab, 560M parameters).

The subtlety is that no labels tell the model which cells are relevant. CABINET treats relevance as a latent variable estimated through variational inference, and shapes the URS representation space with three losses: a clustering loss that groups tokens into relevant and non-relevant clusters, a separation loss that pushes the two cluster centroids apart, and a sparsification loss that drives irrelevant scores toward 0 and relevant scores toward 1. Each token's embedding is multiplied by its relevance score, so noisy cells are softly suppressed rather than deleted, and the whole system trains end-to-end through the answer-generation cross-entropy loss.

The second path supplies an explicit, question-level signal that the URS might miss. The Parsing Statement Generator is a fine-tuned Flan T5-xl, bootstrapped from only about 300 manual annotations. It writes a criterion for the relevant rows and columns, for example "consider rows with result as loss, and note the higher value in the score column," and the cell highlighter uses it to locate the corresponding cells and produce a cell-based score. Where the URS learns an implicit signal, this path adds an explicit one, and the two are complementary.

How the two signals cooperate: a worked example

The figure below uses a question about how many awards "two and a half men" received to show the division of labor. The URS assigns higher scores to the rows that are genuinely relevant (where the show won or was nominated), but it can still miss a row, here the 2006 golden icon award for best actor in a comedy series. The weakly-supervised parsing-statement module fills in exactly those cells the URS overlooked. The t-SNE plot on the lower right reinforces the point: the URS separates the table tokens' latent representations into two clean clusters (red for relevant, blue for non-relevant), evidence that the three clustering-related losses are doing their job.

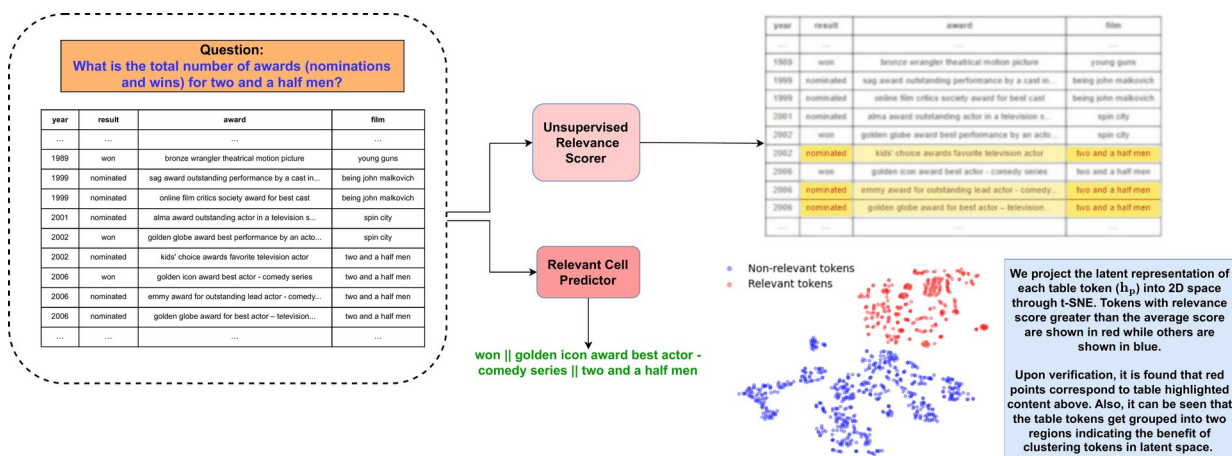


Figure 5. A qualitative view of the two relevance signals. The URS highlights the question-relevant rows (yellow); the parsing-statement module recovers a row the URS missed (2006 golden icon award, best actor, comedy series). The lower-right t-SNE shows the table tokens' latent representations splitting cleanly into relevant (red) and non-relevant (blue) clusters.

Results: new state of the art at 560M parameters

CABINET is evaluated on three challenging table-QA benchmarks: WikiTableQuestions (WikiTQ, which demands compositional reasoning and uses short answers scored by exact-match accuracy), WikiSQL (also scored by exact-match accuracy), and FeTaQA (long, free-form answers scored by Sacre-BLEU). The authors additionally release the roughly 300 manually written parsing statements used to bootstrap the weakly-supervised module.

CABINET sets new state of the art on all three. What stands out is that it does so with a single 560M-parameter model (the OmniTab backbone), while outperforming much larger GPT-3 and Codex based in-context learning methods. On WikiTQ it beats OmniTab, DATER, and a fine-tuned Flan T5-xl by 6.4, 3.2, and 4.7 absolute percentage points respectively.

Table 1. CABINET's headline results across the three benchmarks (all new state of the art).

Benchmark	Metric	CABINET	vs. strongest baseline
WikiTQ	Exact-match accuracy	69.1%	+6.4 over OmniTab
FeTaQA	Sacre-BLEU	40.5	new SoTA
WikiSQL	Exact-match accuracy	89.5%	new SoTA

Not just more accurate, more robust

Soft weighting buys robustness as well as accuracy. The authors perturb the tables in four ways, row addition (RA), row permutation (RP), column permutation (CP), and cell replacement (CR), and measure the relative performance drop. The figure below compares CABINET with OmniTab on WikiTQ: under every perturbation, CABINET degrades markedly less. Under the harshest case, row addition, OmniTab drops 18.89% while CABINET drops only 11.46%. The authors also observe that CABINET keeps its advantage as tables grow larger.

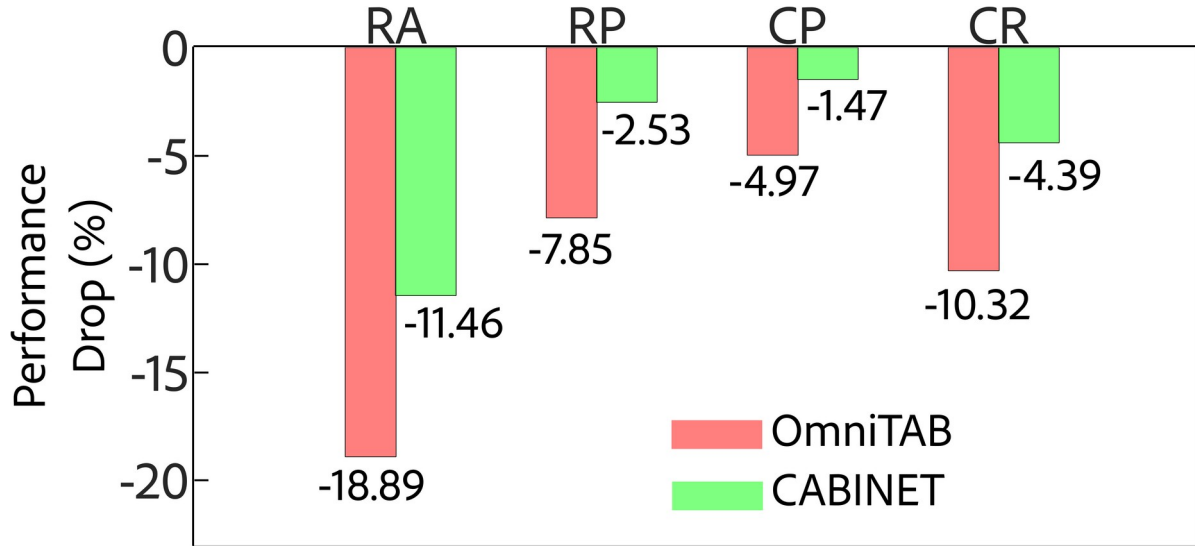


Figure 3. Relative performance drop (%) under table perturbations on WikiTQ. RA is row addition, RP row permutation, CP column permutation, CR cell replacement. CABINET (green) degrades less than OmniTAB (red) under every perturbation, indicating greater robustness to noise.

Ablation: both components pull their weight

Two ablations confirm the design. First, the URS's three losses only help in combination: applying clustering, centroid separation, and sparsification together lifts WikiTQ accuracy from 60.8% to 65.6%, whereas any subset gives little benefit. Second, the two relevance signals need to be fused: weighting the unsupervised score at 0.7 and the cell-based score at 0.3 is optimal (69.1%), while relying on the cell-based signal alone ($\lambda_{\text{uns}}=0$, $\lambda_{\text{cell}}=1$) collapses accuracy to 37.6%. In other words, the unsupervised scorer is the primary driver and the parsing-statement module is a complementary aid.

Table 2. Fusing the two relevance signals on WikiTQ. λ_{uns} weights the unsupervised score, λ_{cell} the cell-based score.

λ_{uns}	λ_{cell}	WikiTQ accuracy
------------------------	-------------------------	-----------------

1.0	0.0	65.6%
0.7	0.3	69.1% (best)
0.5	0.5	68.6%

Takeaway

The core lesson of CABINET is that you do not need to cut a table down to answer questions about it. By learning a relevance score for every cell and weighting softly, instead of hard-decomposing the table and risking the loss of useful information, the model keeps full access to the data while being steered toward what matters. Pairing a differentiable unsupervised relevance scorer with a weakly-supervised parsing-statement cell highlighter is both more accurate and more robust to noise than prior removal-based approaches, setting new state of the art on three table-QA benchmarks with a compact 560M-parameter model.

The approach has its boundaries: the URS learns relevance without supervision, the weakly-supervised module leans on the roughly 300 bootstrapping parsing statements, and the experiments center on English table-QA tasks. But the underlying idea, that soft weighting beats hard deletion, is worth carrying to any structured-reasoning task where a few relevant signals must be found amid noise.